Prescriptive analytics

Optimization

Mathematical optimization (aka *mathematical programming*) is the selection of a ***best*** element from some set of available alternatives.

An optimization problem consists of *minimizing* (*maximizing*) a real function by choosing input values from within an allowed set.

More generally, optimization includes finding ***best available*** values of some *objective function*, given a defined *domain* (or input), where objective functions and domains can be very diverse.

(adapted from Wikipedia)

> NIHIL OMNINO IN MUNDO CONTINGIT, IN QUO NON MAXIMI MINIMIQVE RATIO QUAEPIAM ELUCEAT
>
> *EULER*

Optimization problem

An optimization problem can be represented in the following way:

Given: a function $f: \mathrm{A} \to \mathbb{R}$ from some set A to the real numbers

Find: an element $x_0 \in \mathrm{A}$

- such that $f(x_0) \leq f(x)$ for all $x \in \mathrm{A}$ ("*minimization*") or
- such that $f(x_0) \geq f(x)$ for all $x \in \mathrm{A}$ ("*maximization*").

Subfields

Major subfields

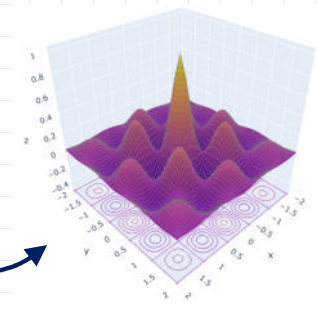
- Linear programming (LP): the objective function f is linear and the constraints are specified using only linear equalities and inequalities.
- Nonlinear programming: the objective function and/or the constraints can be nonlinear.
- Quadratic programming: the objective function can have quadratic terms, while the feasible set must be specified with linear equalities and inequalities.
- Integer programming: linear programs in which some or all variables are constrained to take on integer values (this is in general much more difficult than regular linear programming).
- Combinatorial optimization: problems where the set of feasible solutions is discrete or can be reduced to a discrete one.
- Stochastic programming: some of the constraints or parameters depend on random variables.
- Robust programming: tries to capture uncertainty in the data, finding solutions that are valid under all / a wide set of possible realizations of the uncertainties.
- Constraint programming: a programming paradigm where relations between variables are stated in the form of constraints.

. . . and many more . . . *(Adapted from wikipedia)*

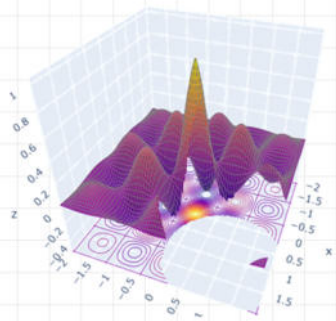
Nonlinear optimization

The objective function or the constraints or both can have nonlinear components.

Unconstrained



Constrained



Example:

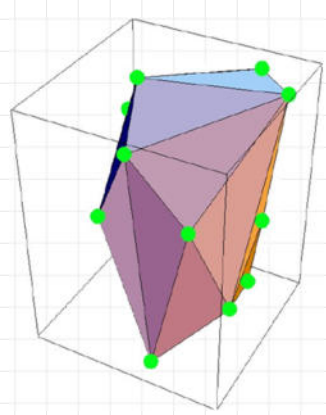
$$\begin{array}{ll} min & x^2-2x+3 \\ s.t. & x^2 \leq 9 \end{array}$$

Linear optimization

- Both the objective function and the constraints are modelled by linear functions.
- Constraints (there have to be some) are in the form of a system of equations or inequalities.

Example:

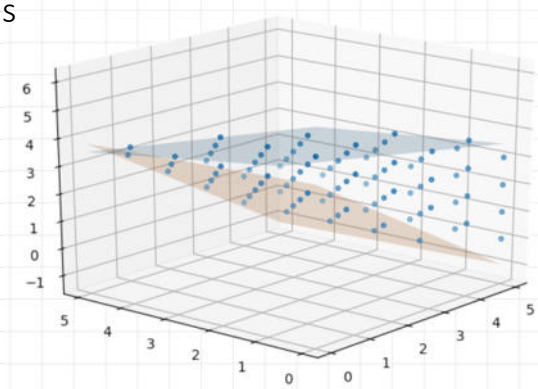
$$\begin{array}{lrrl} min & -x_1 & -x_2 & \\ s.t. & x_1 & -x_2 & \leq 1.5 \\ & -7x_1 & +5x_2 & \leq -17.5 \\ & & -x_2 & \leq -4.5 \\ & x_1, x_2 \geq 0 & & \end{array}$$



Integer linear optimization

Linear optimization problems where (some of) the variables are required to take on integer values.

- If all variables are integer: Integer Linear Programming (ILP)
- If some of the variables are continuous: Mixed Integer Linear programming (MILP).

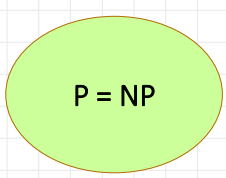


Example:

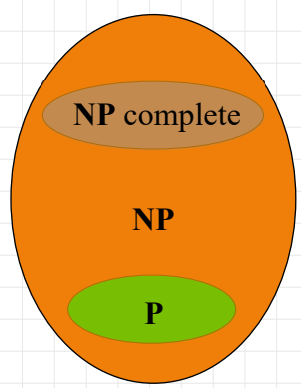
$$
\begin{array}{lrrl}
min & -x_1 & -x_2 & \\
s.t. & x_1 & -x_2 & \leq 1.5 \\
 & -7x_1 & +5x_2 & \leq -17.5 \\
 & & -x_2 & \leq -4.5 \\
 & \multicolumn{3}{l}{x_1, x_2 \geq 0 \ and \ integer}
\end{array}
$$

Theory: P and NP

P = NP



P ≠ NP



Computational issues

- The size of the instances met in real world applications rules out the possibility of solving them to optimality in an acceptable time.
- Nevertheless, these instances must be solved.
- Thus the need to look for suboptimal solutions, provided that they are of "*acceptable quality*" and that they can be found in "*acceptable time*".

A wide scope

A combinatorial optimization problem is defined over a set $C = \{c_1, \ldots, c_n\}$ of basic components.

A solution of the problem is a subset $S \subset C$;

$F \subseteq 2^C$ is the subset of *feasible solutions*, (a solution S is feasible iff $S \in F$).

$z: 2^C \rightarrow \Re$ is the cost function,

The objective is to find a minimum cost feasible solution $S°$, i.e., to find $S° \in F$ such that $z(S°) \leq z(S)$, $\forall S \in F$.

Failing this, the algorithm anyway returns the best feasible solution found, $S^* \in F$.

Example: Travelling Salesman Problem

n locations given
(home of salesperson and n-1 customers).

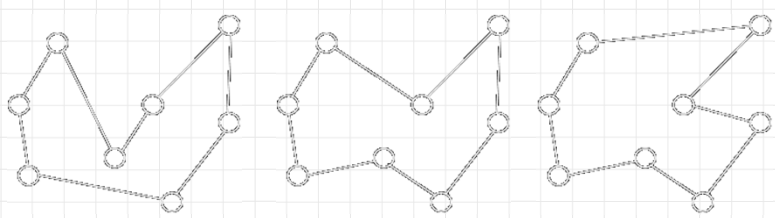
$[d_{ij}]$ distance matrix given, contains distances (Km, time, …)
between each pair of locations.

The TSP asks for the shortest tour starting home, visiting each customer once and returning home

Symmetric / asymmetric versions.

TSP

Which is the shortest tour?



Difficult to say …

n nodes $\Rightarrow$ (n-1)! possible tours

n	5	10	15	20
n!	120	3,628.800	1,31*10^12	2,43*10^18

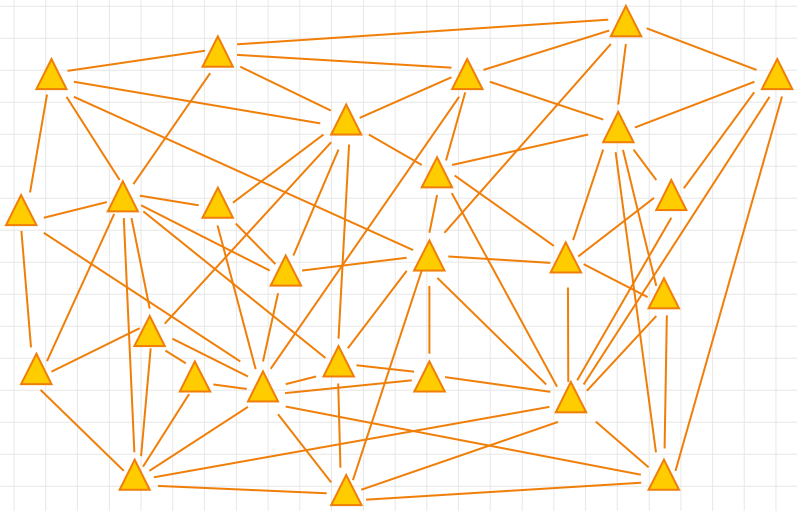
TSP as a CO example

TSP is defined over a weighted undirected complete graph G=(V,E,D), where V is the set of vertices, E is the set of edges and D is the set of edge weights.

- The component set corresponds to E (C=E),
- F corresponds to the set of Hamiltonian cycles in G
- $z(S)$ is the sum of the weights associated to the edges belonging to the solution S.

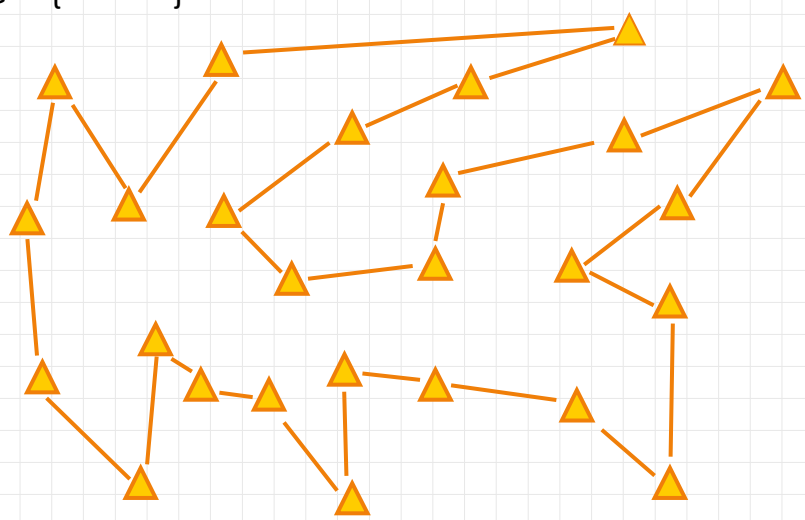
TSP: component set (part of)

C = { ——— }



TSP: a solution

C = { ——— }



LP-Formulation

LP formulation is a binary LP (IP) which is almost identical to the assignment problem:

$$x_{ij} = \begin{cases} 1 & \text{if } j \text{ is visited immediately after } i \\ 0 & \text{otherwise} \end{cases}$$

$$min \sum_{i=1}^{n}\sum_{j=1}^{n} c_{ij}x_{ij}$$ minimize total distance travelled

$\sum_{i=1}^{n} x_{ij} = 1$ for $j = 1, \dots, n$ — enter each city j once

$\sum_{j=1}^{n} x_{ij} = 1$ for $i = 1, \dots, n$ — leave each city i once

$x_{ij} \in \{0, 1\}$ — binary variables

plus conditions to avoid short cycles

$\sum_j x_{ij} = 2$, $i=1, \dots, n$ for undirected problems.

Generalized assignment: math model

The problem can be modeled on a bipartite complete directed graph $G(I \cup J, A)$, where the *vertices* in I denote the potential facilities, the vertices in J the customers, and the *arcs* in $A = I \times J$ are associated with the material flows between the potential facilities and the demand points.

Demand could be either divisible or not. Let:

- Q_i $i \in I$, the capacity of the facility i;
- q_{ij} $j \in J$, be the demand of customer j when serviced by facility i;
- x_{ij} $i \in I$, $j \in J$, a decision variable representing the (<u>*percentual*</u>) amount of product sent from site i to demand point j;
- c_{ij} $i \in I$, $j \in J$, the cost of transporting all products from site i to customer j;

Math model:(MIP)

$$z(GAP) = min \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} \qquad (1)$$

$$subject\ to \sum_{i \in I} x_{ij} = 1 \qquad j \in J \qquad (2)$$

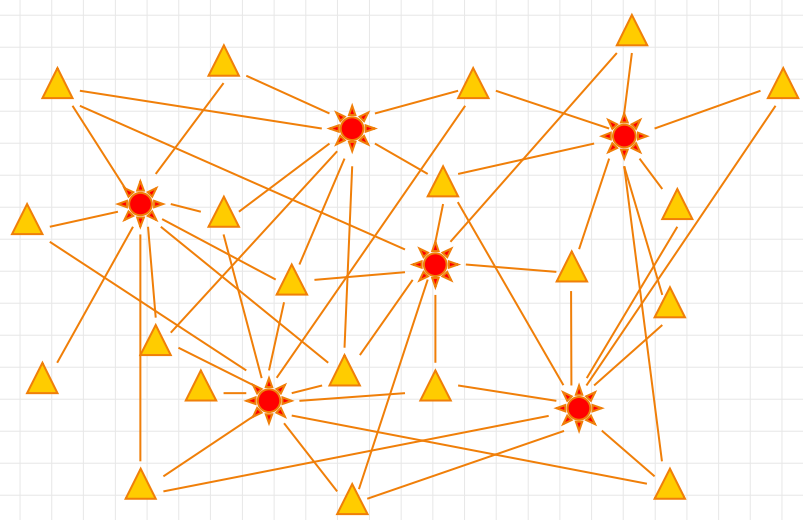
$$\sum_{j \in J} q_{ij} x_{ij} \leq Q_i \qquad i \in I \qquad (3)$$

$$x_{ij} \in \{0,1\} \qquad i \in I, j \in J \qquad (4)$$

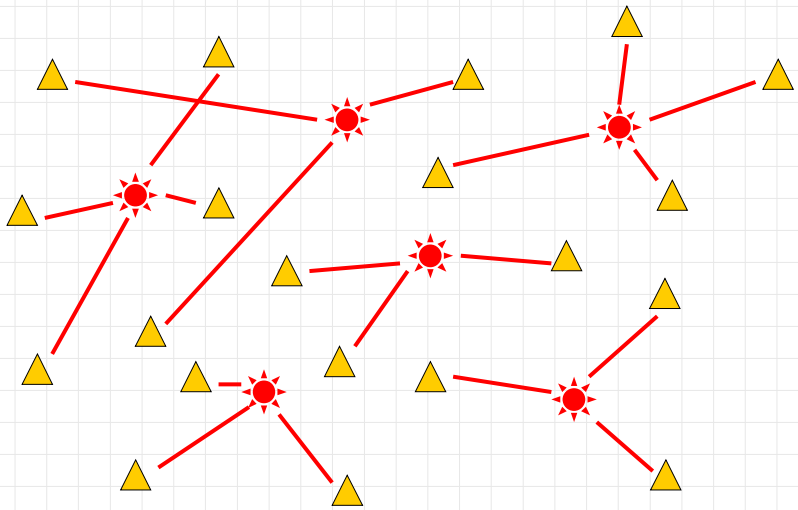
Math model

- Variables x_{ij}. $i \in \mathrm{I}, j \in \mathrm{J}$. determine customer allocations to facilities.
- The objective function (1) is the cost for transporting goods from facilities to customers.
- Constraints (2) ensure that each customer request is satisfied.
- Constraints (3) force the total request to a facility does not exceed the corresponding capacity.
- Constraints (4) could be either $x_{ij} \geq 0$, clients service can be split, or $x_{ij} \in \{0,1\}$, clients are singly assigned to a warehouse.

GAP: component set (part of)



GAP: solution



Constructive heuristics

```
function Constructive()
1. Sort the components in C by increasing costs.
2. Set S*=∅ and i=1.
3. Repeat
      if (S* ∪ c_i is a feasible partial solution)
          then S* = S* ∪ c_i.
      i=i+1.
   until S* ∈ F.
```

Constructive heuristics

A constructive approach can yield optimal solutions for certain problems, e.g., the MST.

In other cases it could be unable to construct a feasible solution.

TSP: order all edges by increasing edge cost, take the least cost one and add to it increasing cost edges, provided they do not close subtours, until a Hamiltonian circuit is completed.

More involved constructive strategies give rise to well-known TSP heuristics, like the Farthest Insertion, the Nearest Neighbor, the Best Insertion or the Sweep algorithms.

Nearest Neighbor Heuristic

```
function NearestNeighbor()
0. Start from any city; e.g. the first one
1. Find nearest neighbor (to the last city) not already visited
2. repeat 1 until all cities are visited.
   then connect last city with starting point
```

Ties can be broken arbitrarily.

Starting from different starting points gives different solutions! Some will be bad some will be good.

GREEDY ⇒ the last connections tend to be long and improvement heuristics should be used afterwards

Local Search: neighborhoods

The neighborhood of a solution S, N(S), is a subset of 2^C defined by a *neighborhood function* N: $2^C \rightarrow 2^{2^C}$.

Often only feasible solutions considered, thus neighborhood functions N: $F \rightarrow 2^F$.

The specific function used has a deep impact on the algorithm performance and its choice is left to the algorithm designer.

Local search

```
function LocalSearch()
1. Generate an initial feasible solution S.
2. Find S'∈N(S), such that z(S')=min z(S ), ∀S ∈ N(S).
3. If z(S') < z(S) then S=S'
   go to step 2.
4. S* = S.
```

The update of a solution in step 3 is called a *move* from S to S'.

It could be made to the first improving solution found.

Local search

There are problems for which a local search approach guarantees to find an optimal solution (ex. the simplex algorithm).

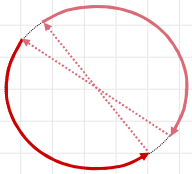
For TSP, two LS are 2-opt and 3-opt, which take a solution (a list of *n* vertices) and exhaustively swap the elements of all pairs or triplets of arcs in it.

More sophisticated neighborhoods give rise to more effective heuristics, among which Lin and Kernighan [LK73].

Local search heuristics

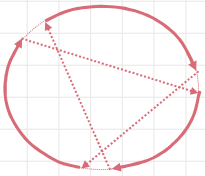
2 – opt (arcs): try *all* inversions of some subsequence

if improvement ⇒ start again from beginning



3 – opt (arcs): try *all* shifts of some subsequence to different positions

if improvement ⇒ start again from beginning



2-opt (nodes): swap every pair of nodes in the permutation representing the solution.

if improvement ⇒ start again from beginning

3-opt (nodes): swap every triplet of nodes in the permutation representing the solution.

if improvement ⇒ start again from beginning

Metaheuristics: focus on heuristic guidance

Simple heuristics can perform very well, but can get trapped in poor quality solutions (local optima).

New approaches have been presented starting from the mid '70ies.

They are algorithms which manage the trade-off between search diversification, when search is going on in bad regions of the search space, and intensification, aimed at finding the best solutions within the region being analyzed.

These algorithms have been named metaheuristics.

Metaheuristics

"A master strategy that guides and modifies other heuristics to produce solutions beyond those that are normally generated in a quest for local optimality" (Glover, Laguna, 1997).

"An iterative master process that guides and modifies the operations of subordinate heuristics to efficiently produce high-quality solutions.
It may manipulate a complete (or incomplete) single solution or a collection of solutions at each iteration.
The subordinate heuristics may be high (or low) level procedures, or a simple local search, or just a construction method.
The family of meta-heuristics includes, but is not limited to, adaptive memory procedures, tabu search, ant systems, greedy randomized adaptive search, variable neighborhood search, evolutionary methods, genetic algorithms, scatter search, neural networks, simulated annealing, and their hybrids". (Voß et al.,1999)

Metaheuristics

All relevant metaheuristics have been hybridized with MP, generating matheuristics (red ones detailed later):

- Iterated local search
- Simulated annealing
- Tebu search
- GRASP
- Variable neighborhood search
- ALNS, CMSA, ejection chains
- Genetic algorithms
- Evolution strategies
- Ant colony optimization
- Scatter search
- Particle swarm optimization
- …

Iterated Local Search

Repeat local searches starting from different initial solutions.

Initial solutions should not be random to increase effectiveness.

Bespoke perturbations of previous solutions or of problem data work best.

```
function IteratedLocalSearch()
1. Generate an initial feasible solution x0;
2. Apply a local search on x0 obtaining a local optimum x∘;
3. repeat
4.    Set x  = Perturbation(x∘, history);
5.    Set x  = LocalSearch(x);
6 .   Set x∘ = AcceptanceCriterion(x∘, x, history);
7.    if(z(x∗) > z(x∘)) then set x∗ = x∘;
8. until termination condition;
9. return x∗
```

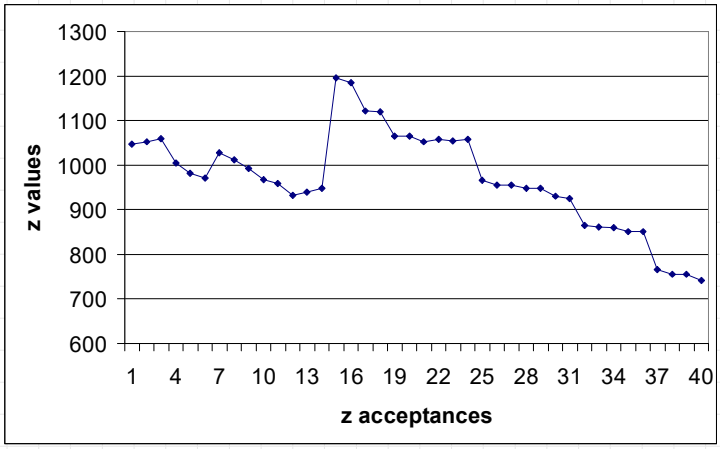
Simulated Annealing

Simulated Annealing, a simple and widely used metaheuristic.

```
function SimulatedAnnealing(T)
1  Generate a feasible solution x; Set x* = x;
2  Generate a feasible solution x' ∈ N(x);
3  if (z(x') < z(x)) then set x = x';
4  if (z(x*) > z(x)) then x* = x;
5  else set x = x' with probability p = exp-(z(x')-z(x))/kT;
6  if (annealing condition) then decrease T;
7  if not(terminating condition) then go to 2;
8  return x*;
```


SA example: ulysses 16

Simulated annealing trace



Tabu search

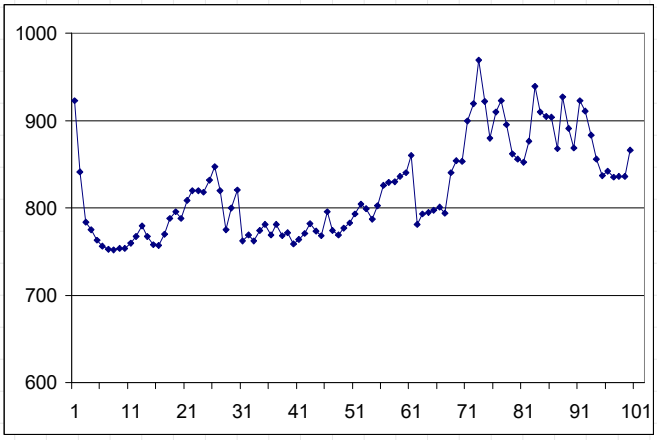
Tabu Search (TS) [GL97] escapes from local minima by moving onto the best solution of the neighborhood at each iteration, even though it is worse than the current one.

A memory structure called *tabu list*, TL, forbids to return to already explored solutions.

```
function TabuSearch()
1.  Generate an initial feasible solution S,
    set S* = S and initialize TL=∅.
2.  Find S' ∈N(S), such that
    z(S')=min {z(S ), ∀S ∈ N(S), S ∉ TL}.
3.  S=S', TL=TL ∪ {S}, if (z(S*) > z(S)) set S* = S.
4.  If not(end condition) go to step 2.
```


TS example: ulysses 16

Tabu Search trace



GRASP

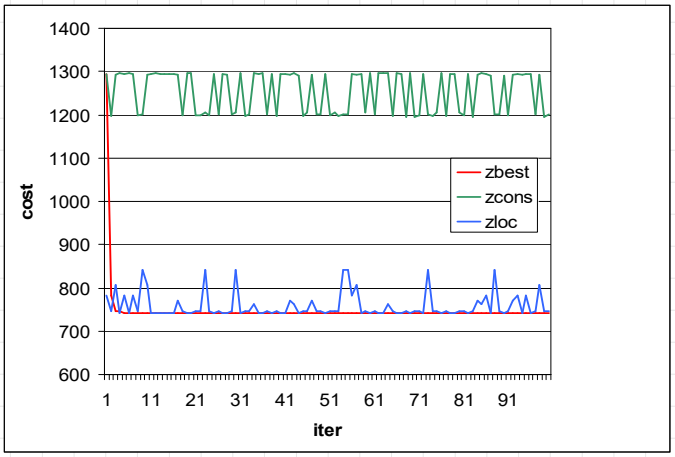
GRASP (Greedy Randomized Adaptive Search Procedure) restarts search from another promising region of the search space as soon as a local optimum is reached.

GRASP consists in a multistart approach with a suggestion on how to construct the initial solutions.

```
function GRASP()
1. Build a solution S (=S* in the first iteration) by a constructive
   greedy randomized procedure based on candidate lists.
2. Apply a local search procedure starting from S and  producing S'.
3. If z(S')<z(S*) then S*=S'.
4. If not(end condition) go to step 1.
```


GRASP example: ulysses 16

GRASP trace



Variable Neighborhood Search

Variable Neighborhood Search (VNS) explores a sequence of neighborhoods at each iteration (called a Variable Neighborhood Descent, VND), followed by a perturbation of the incumbent solution.

A local minimum with respect to one neighborhood is not necessarily a local minimum for other neighborhoods. Only a global optimum is a local optimum for any neighborhood.

The general structure of the VND starts from a seed solution, then applies to it all chosen neighborhoods in a fixed order.

Each time an improvement is found, the corresponding local optimum becomes the new incumbent solution and the process starts again from the beginning.

VNS adds a perturbation step and an acceptance step.

- The perturbation step at the end of VND re-instantiates search on another seed solution.
- The acceptance step allows the evaluation of search exploration versus exploitation (perturbed sol or better minimum from the old one).

Example, metaH hybrid, VNS

```
function VariableNeighborhoodDescent(x)
Define Nk, k = 1, ... , kmax;                // the neighborhoods
Set k = 1;
repeat
  Set x' = FindBestNeighbor(x, Nk);
  if (z(x') < z(x)) then x = x'; x* = x; k = 1;
  else k = k + 1;
until k > kmax;
return x*;

function VariableNeighborhoodSearch()
Generate an initial feasible solution x;
Set x* = x;
repeat
  Set x' = VariableNeighborhoodDescent(x) ; // can be any local search
  if (z(x*) > z(x')) then set x* = x';
  Set x = Perturb(x');                       // perturbation
  Set x = Accept(x,history);                 // acceptance
until termination condition;
return x*;
```

ALSN

Adaptive Large Neighborhood Search maintains sets of destroy and reconstruct operators, records the performance of each, and dynamically adjusts the weights used to choose (in probability or otherwise) which method to use at each iteration.

```
function ALSN()
// Given: an initial solution
//        a set of destroy methods Ω- and associated weights w-
//        a set of  repair methods Ω+ and associated weights w+
2 Repeat until stop-criterion:
    Choose a method from Ω- and a method from Ω+
    Destroy part of solution
    Repair solution
    Accept or reject new solution
    Update destroy and repair weights w- and w+
3 Return best solution encountered
```